

hwk2/tb4_branch_system.vhd

```

1  -----
2  -- TB4_BranchSystem - SH-2 CPU Integration Testbench 4
3  --
4  -- Two phases controlled by a single signal "loading":
5  --   loading='1' : TB owns the bus (load hex, then check memory)
6  --               CPU is held in reset so it tri-states all its outputs
7  --   loading='0' : CPU owns the bus (program execution)
8  --               TB drives 'Z' so it doesn't interfere
9  --
10 -- CPU, memory, and TB all share the same resolved bus signals.
11 -- Only one non-Z driver is active at any given time. PC relative offsets
12 -- will need to be changed when pipelining.
13 --
14 -- Instructions covered (coherent real-program narrative):
15 --
16 --   BRA label          - delayed unconditional branch; delay slot verified
17 --   BSR label / RTS    - delayed subroutine call and return
18 --   BT label (taken)   - branch when T=1
19 --   BT label (not taken) - no branch when T=0; fall-through verified
20 --   BF label (taken)   - branch when T=0
21 --   BF label (not taken) - no branch when T=1; fall-through verified
22 --   BRAF Rm            - delayed far branch (PC-relative via register)
23 --   LDC Rm,GBR         - load GBR from register
24 --   STC GBR,Rn         - store GBR to register
25 --   LDS Rm,PR / STS PR,Rn - PR roundtrip
26 --   CLRT / SETT       - T-bit manipulation (precondition for BT/BF)
27 --   STC SR,Rn         - read SR; T=0 bit verified
28 --   STC.L GBR,@-Rn     - push GBR onto stack
29 --   LDC.L @Rm+,GBR     - pop GBR from stack
30 --   STS.L PR,@-Rn      - push PR onto stack
31 --   LDS.L @Rm+,PR      - pop PR from stack
32 --   MOV.L @(disp,PC),Rn - constant loading
33 --   MOV.L Rm,@Rn / ADD#4 - result store / pointer advance
34 --   NOP               - delay slot filler
35 --
36 -- Methodology:
37 --   Each test section stores a unique "breadcrumb" value at the next
38 --   result slot in the data area. Wrong control flow writes the wrong
39 --   value (or skips the write entirely), which the check phase detects.
40 --   Delay-slot execution is verified by having the slot instruction
41 --   increment R13; the incremented value is then stored and checked.
42 --
43 -- Expected results in data memory (block 1):
44 --
45 --   0x1000 0x00000001 BRA destination reached
46 --   0x1004 0x00000001 BRA delay slot verified (R13=1, MOVT)
47 --   0x1008 0x00000002 BT taken (T=1)
48 --   0x100C 0x00000003 BT not-taken fall-through (T=0)
49 --   0x1010 0x00000004 BF taken (T=0)
50 --   0x1014 0x00000005 BF not-taken fall-through (T=1)
51 --   0x1018 0x00000006 BRAF destination reached
52 --   0x101C 0x00000003 BSR subroutine body
53 --   0x1020 0x00000004 RTS: returned to caller
54 --   0x1024 0x00001200 LDC/STC GBR roundtrip
55 --   0x1028 0xDEADC0DE LDS/STS PR roundtrip
56 --   0x102C SR value    STC SR: T bit (bit 0) = 0 after CLRT
57 --   0x1030 0x00001200 STC.L/LDC.L GBR stack roundtrip
58 --   0x1034 0xDEADC0DE STS.L/LDS.L PR stack roundtrip
59 --
60 -- Program image (assembled; 49 words = 196 bytes):
61 --
62 -- Disassembly (address : encoding : instruction):
63 -- -- Setup --
64 --   0x0000: DE2D  MOV.L @(45,PC),R14    R14 = 0x00001000
65 --   0x0002: 0009  NOP

```

```

66 --      0x0004: DF2D  MOV.L @(45,PC),R15  R15 = 0x000011F0
67 --      0x0006: ED00  MOV #0,R13          R13 = 0
68 --  -- T4-1: BRA + delay slot --
69 --      0x0008: A000  BRA +0              -> 0x000C
70 --      0x000A: 7D01  ADD #1,R13          [delay slot]
71 --      0x000C: E101  MOV #1,R1
72 --      0x000E: 2E12  MOV.L R1,@R14       [0x1000] = 1
73 --      0x0010: 7E04  ADD #4,R14
74 --      0x0012: E201  MOV #1,R2
75 --      0x0014: 3D20  CMP/EQ R2,R13      T=1 if delay ran
76 --      0x0016: 0329  MOVT R3
77 --      0x0018: 2E32  MOV.L R3,@R14       [0x1004] = 1
78 --      0x001A: 7E04  ADD #4,R14
79 --  -- T4-2: BT taken (T=1) --
80 --      0x001C: 0018  SETT
81 --      0x001E: E102  MOV #2,R1
82 --      0x0020: 8901  BT +1              -> 0x0026 (taken)
83 --      0x0022: E1EE  MOV #0xEE,R1       (poison)
84 --      0x0024: 0009  NOP
85 --      0x0026: 2E12  MOV.L R1,@R14       [0x1008] = 2
86 --      0x0028: 7E04  ADD #4,R14
87 --  -- T4-3: BT not-taken (T=0) --
88 --      0x002A: 0008  CLRT
89 --      0x002C: E103  MOV #3,R1
90 --      0x002E: 8901  BT +1              -> 0x0034 (not taken)
91 --      0x0030: 2E12  MOV.L R1,@R14       [0x100C] = 3
92 --      0x0032: 7E04  ADD #4,R14
93 --  -- T4-4: BF taken (T=0) --
94 --      0x0034: 0008  CLRT
95 --      0x0036: E104  MOV #4,R1
96 --      0x0038: 8B01  BF +1              -> 0x003E (taken)
97 --      0x003A: E1DD  MOV #0xDD,R1       (poison)
98 --      0x003C: 0009  NOP
99 --      0x003E: 2E12  MOV.L R1,@R14       [0x1010] = 4
100 --      0x0040: 7E04  ADD #4,R14
101 --  -- T4-5: BF not-taken (T=1) --
102 --      0x0042: 0018  SETT
103 --      0x0044: E105  MOV #5,R1
104 --      0x0046: 8B01  BF +1              -> 0x004C (not taken)
105 --      0x0048: 2E12  MOV.L R1,@R14       [0x1014] = 5
106 --      0x004A: 7E04  ADD #4,R14
107 --  -- T4-6: BRAF --
108 --      0x004C: E104  MOV #4,R1           offset = 4
109 --      0x004E: E206  MOV #6,R2
110 --      0x0050: 0123  BRAF R1             -> 0x0058
111 --      0x0052: 0009  NOP                 [delay slot]
112 --      0x0054: E2BB  MOV #0xBB,R2       (poison)
113 --      0x0056: 0009  NOP
114 --      0x0058: 2E22  MOV.L R2,@R14       [0x1018] = 6
115 --      0x005A: 7E04  ADD #4,R14
116 --  -- T4-7: BSR + RTS --
117 --      0x005C: B005  BSR +5              -> 0x006A (subroutine)
118 --      0x005E: 0009  NOP                 [delay slot]
119 --      0x0060: E104  MOV #4,R1           (return point)
120 --      0x0062: 2E12  MOV.L R1,@R14       [0x1020] = 4
121 --      0x0064: 7E04  ADD #4,R14
122 --      0x0066: A005  BRA +5              -> 0x0074 (skip sub)
123 --      0x0068: 0009  NOP                 [delay slot]
124 --  -- subroutine --
125 --      0x006A: E103  MOV #3,R1
126 --      0x006C: 2E12  MOV.L R1,@R14       [0x101C] = 3
127 --      0x006E: 7E04  ADD #4,R14
128 --      0x0070: 000B  RTS
129 --      0x0072: 0009  NOP                 [delay slot]
130 --  -- T4-8: LDC/STC GBR --
131 --      0x0074: D112  MOV.L @(17,PC),R1   R1 = 0x1200
132 --      0x0076: 411E  LDC R1,GBR

```

```

133 -- 0x0078: 0212 STC GBR,R2
134 -- 0x007A: 2E22 MOV.L R2,@R14      [0x1024] = 0x1200
135 -- 0x007C: 7E04 ADD #4,R14
136 -- -- T4-9: LDS/STS PR --
137 -- 0x007E: D111 MOV.L @(17,PC),R1   R1 = 0xDEADC0DE
138 -- 0x0080: 412A LDS R1,PR
139 -- 0x0082: 022A STS PR,R2
140 -- 0x0084: 2E22 MOV.L R2,@R14      [0x1028] = 0xDEADC0DE
141 -- 0x0086: 7E04 ADD #4,R14
142 -- -- T4-10: CLRT + STC SR --
143 -- 0x0088: 0008 CLRT
144 -- 0x008A: 0102 STC SR,R1
145 -- 0x008C: 2E12 MOV.L R1,@R14      [0x102C] = SR (T=0)
146 -- 0x008E: 7E04 ADD #4,R14
147 -- -- T4-11: STC.L/LDC.L GBR stack roundtrip --
148 -- 0x0090: 4F13 STC.L GBR,@-R15
149 -- 0x0092: E100 MOV #0,R1
150 -- 0x0094: 411E LDC R1,GBR          (clear GBR)
151 -- 0x0096: 4F17 LDC.L @R15+,GBR
152 -- 0x0098: 0112 STC GBR,R1
153 -- 0x009A: 2E12 MOV.L R1,@R14      [0x1030] = 0x1200
154 -- 0x009C: 7E04 ADD #4,R14
155 -- -- T4-12: STS.L/LDS.L PR stack roundtrip --
156 -- 0x009E: D109 MOV.L @(8,PC),R1   R1 = 0xDEADC0DE
157 -- 0x00A0: 412A LDS R1,PR
158 -- 0x00A2: 4F22 STS.L PR,@-R15
159 -- 0x00A4: E100 MOV #0,R1
160 -- 0x00A6: 412A LDS R1,PR          (clear PR)
161 -- 0x00A8: 4F26 LDS.L @R15+,PR
162 -- 0x00AA: 012A STS PR,R1
163 -- 0x00AC: 2E12 MOV.L R1,@R14      [0x1034] = 0xDEADC0DE
164 -- 0x00AE: 7E04 ADD #4,R14
165 -- -- End --
166 -- 0x00B0: AFFE BRA .               self-loop
167 -- 0x00B2: 0009 NOP
168 -- -- Constant pool --
169 -- 0x00B4: 00001000                data_base
170 -- 0x00B8: 000011F0                stack_base
171 -- 0x00BC: 00001200                gbr_val
172 -- 0x00C0: DEADC0DE                pr_magic

```

```

173 -----
174
175 library ieee;
176 use ieee.std_logic_1164.all;
177 use ieee.numeric_std.all;
178 use std.textio.all;
179
180 entity TB4_BranchSystem is
181 end entity;
182
183 architecture sim of TB4_BranchSystem is
184
185     constant CLK_PERIOD : time      := 20 ns;
186     constant PROG_HEX   : string    := "tb4_program.hex";
187     constant MEM_WORDS  : integer   := 256;
188
189     signal clk          : std_logic := '0';
190     signal reset        : std_logic := '0';
191     signal loading      : std_logic := '1';
192
193     -- Shared bus -- CPU, memory, and TB all connect here
194     signal AB : std_logic_vector(31 downto 0);
195     signal DB : std_logic_vector(31 downto 0);
196     signal RE0, RE1, RE2, RE3 : std_logic;
197     signal WE0, WE1, WE2, WE3 : std_logic;
198
199     -- TB-side bus signals

```

```

200     signal tb_AB : std_logic_vector(31 downto 0) := (others => '0');
201     signal tb_DB : std_logic_vector(31 downto 0) := (others => '0');
202     signal tb_RE : std_logic := '1';
203     signal tb_WE : std_logic := '1';
204
205 begin
206
207     clk <= not clk after CLK_PERIOD / 2;
208
209     -- TB bus mux: drives bus when loading='1', releases ('Z') when loading='0'
210     -- During loading='0' the CPU drives the bus instead (via its port map)
211     AB <= tb_AB when loading = '1' else (others => 'Z');
212     DB <= tb_DB when (loading = '1' and tb_WE = '0') else (others => 'Z');
213     RE0 <= tb_RE when loading = '1' else 'Z';
214     RE1 <= tb_RE when loading = '1' else 'Z';
215     RE2 <= tb_RE when loading = '1' else 'Z';
216     RE3 <= tb_RE when loading = '1' else 'Z';
217     WE0 <= tb_WE when loading = '1' else 'Z';
218     WE1 <= tb_WE when loading = '1' else 'Z';
219     WE2 <= tb_WE when loading = '1' else 'Z';
220     WE3 <= tb_WE when loading = '1' else 'Z';
221
222     -- CPU connects directly to the shared bus
223     dut : entity work.SH2_CPU
224         port map (
225             Reset => reset, NMI => '1', INT => '1', clock => clk,
226             AB => AB, DB => DB,
227             RE0 => RE0, RE1 => RE1, RE2 => RE2, RE3 => RE3,
228             WE0 => WE0, WE1 => WE1, WE2 => WE2, WE3 => WE3
229         );
230
231     mem : entity work.MEMORY32x32
232         generic map (
233             MEMSIZE => MEM_WORDS,
234             START_ADDR0 => 16#00000000#,
235             START_ADDR1 => 16#00001000#,
236             START_ADDR2 => 16#10000000#,
237             START_ADDR3 => 16#20000000#
238         )
239         port map (
240             RE0 => RE0, RE1 => RE1, RE2 => RE2, RE3 => RE3,
241             WE0 => WE0, WE1 => WE1, WE2 => WE2, WE3 => WE3,
242             MemAB => AB, MemDB => DB
243         );
244
245     stim : process
246         file hex_file : text;
247         variable fstatus : file_open_status;
248         variable l : line;
249         variable word : std_logic_vector(31 downto 0);
250         variable addr : unsigned(31 downto 0);
251         variable pass_cnt : integer := 0;
252         variable fail_cnt : integer := 0;
253
254         procedure check(a : std_logic_vector(31 downto 0);
255             exp : std_logic_vector(31 downto 0);
256             tag : string) is
257         begin
258             tb_AB <= a;
259             tb_RE <= '0';
260             tb_WE <= '1';
261             wait for CLK_PERIOD;
262             if DB = exp then
263                 pass_cnt := pass_cnt + 1;
264                 report "PASS: " & tag severity note;
265             else
266                 fail_cnt := fail_cnt + 1;

```

```

267         report "FAIL: " & tag
268             & " got=0x" & to_hstring(DB)
269             & " exp=0x" & to_hstring(exp) severity error;
270     end if;
271     tb_RE <= '1';
272     wait for CLK_PERIOD;
273 end procedure;
274
275 procedure check_tbit_zero(a : std_logic_vector(31 downto 0);
276                          tag : string) is
277 begin
278     tb_AB <= a;
279     tb_RE <= '0';
280     tb_WE <= '1';
281     wait for CLK_PERIOD;
282     if DB(0) = '0' then
283         pass_cnt := pass_cnt + 1;
284         report "PASS: " & tag severity note;
285     else
286         fail_cnt := fail_cnt + 1;
287         report "FAIL: " & tag
288             & " SR=0x" & to_hstring(DB)
289             & " bit[0]=T expected 0, got 1" severity error;
290     end if;
291     tb_RE <= '1';
292     wait for CLK_PERIOD;
293 end procedure;
294
295 begin
296     -----
297     -- LOAD PHASE
298     -----
299     loading <= '1';
300     reset   <= '0';
301     tb_RE   <= '1';
302     tb_WE   <= '1';
303     wait for CLK_PERIOD * 2;
304
305     file_open(fstatus, hex_file, PROG_HEX, read_mode);
306     assert fstatus = open_ok
307         report "Cannot open " & PROG_HEX severity failure;
308
309     addr := (others => '0');
310     while not endfile(hex_file) loop
311         readline(hex_file, l);
312         if l'length > 0 then
313             hread(l, word);
314             tb_AB <= std_logic_vector(addr);
315             tb_DB <= word;
316             tb_WE <= '0';
317             wait for CLK_PERIOD;
318             tb_WE <= '1';
319             wait for CLK_PERIOD;
320             addr := addr + 4;
321         end if;
322     end loop;
323     file_close(hex_file);
324
325     -----
326     -- CPU PHASE
327     -----
328     wait until rising_edge(clk);
329     loading <= '0';
330     reset   <= '1';
331
332     wait for CLK_PERIOD * 2;
333

```

```

334     for i in 1 to 600 loop
335         wait until rising_edge(clk);
336     end loop;
337
338     -----
339     -- CHECK PHASE
340     -----
341     wait until rising_edge(clk);
342     reset    <= '0';
343     loading  <= '1';
344     tb_RE    <= '1';
345     tb_WE    <= '1';
346     wait for CLK_PERIOD * 2;
347
348     check(x"00001000", x"00000001", "BRA destination reached");
349     check(x"00001004", x"00000001", "BRA delay slot ran (R13=1)");
350     check(x"00001008", x"00000002", "BT taken (T=1)");
351     check(x"0000100C", x"00000003", "BT not-taken fall-through (T=0)");
352     check(x"00001010", x"00000004", "BF taken (T=0)");
353     check(x"00001014", x"00000005", "BF not-taken fall-through (T=1)");
354     check(x"00001018", x"00000006", "BRAf destination reached");
355     check(x"0000101C", x"00000003", "BSR subroutine body");
356     check(x"00001020", x"00000004", "RTS: returned to caller");
357     check(x"00001024", x"00001200", "LDC/STC GBR roundtrip");
358     check(x"00001028", x"DEADC0DE", "LDS/STS PR roundtrip");
359     check_tbit_zero(x"0000102C", "STC SR: T=0 after CLRT");
360     check(x"00001030", x"00001200", "STC.L/LDC.L GBR stack roundtrip");
361     check(x"00001034", x"DEADC0DE", "STS.L/LDS.L PR stack roundtrip");
362
363     -----
364     -- SUMMARY
365     -----
366     report "TB4: PASSED=" & integer'image(pass_cnt)
367         & " FAILED=" & integer'image(fail_cnt) severity note;
368     if fail_cnt = 0 then
369         report "*** ALL TESTS PASSED ***" severity note;
370     else
371         assert false
372         report integer'image(fail_cnt) & " TEST(S) FAILED"
373             severity failure;
374     end if;
375     wait;
376 end process;
377
378 end architecture;
379

```